

What quantifier elimination can do

Verified procedures that turn a formula with quantifiers over the reals into an equivalent formula without them, and a proof command that uses them inside PVS proofs: the theorems, fourteen examples with the answers as the prover prints them, and what is and is not covered.

In one line. Give `qe8` a formula such as $\exists t. t^2 + at + b = 0$ and it computes a formula in a, b alone, here a case split on the sign of b that reads $b < 0 \vee b = 0 \vee (b > 0 \wedge a^2 - 4b \geq 0)$, which is equivalent to $a^2 \geq 4b$. That the computed formula has no quantifiers and holds at exactly the same points as the input is a **theorem proved in PVS**: `qe8_complete` (and `qe_complete` for `qe`) for every prenex formula with at least one quantifier and one or more free variables, and `qelim_ok` with `qelim_qf` for every first-order formula, with its quantifiers anywhere and any number of free variables, none included.

What is proved

`qe8.qe8_complete` (Tarski–Seidenberg, computed)

```
qe8_complete: THEOREM cons?(os) AND length(pt) = m IMPLIES
  qf?(qe8(m, os, F, phi)`out) AND
  (qfsem(qe8(m, os, F, phi)`out)(pt) IFF fsem(os, F, phi, pt))
```

In words: for every number $m \geq 1$ of free variables (m is a `posnat`), every prenex formula

$$Q_1 x_1 Q_2 x_2 \cdots Q_n x_n. \varphi(y_1, \dots, y_m, x_1, \dots, x_n) \quad (n \geq 1, Q_i \in \{\forall, \exists\})$$

whose matrix φ is any Boolean combination of sign conditions ($< 0, = 0, > 0$, and so $\leq, \geq, \neq$) on polynomials with rational coefficients, and every point $y \in \mathbb{R}^m$: the formula `qe8` computes is quantifier-free and holds at y exactly when the input formula does. The hypotheses only fix the shape (at least one quantifier, `cons?(os)`; a point with m coordinates): `qe8` always succeeds and always returns a quantifier-free formula. `qe8` runs the Collins CAD ([docs/collins_cad.pdf](#)); `qe_all`'s `qe_complete` states the same for `qe`, the first procedure, which runs the read-closure engine.

`qelim_ok.qelim_ok` (every first-order formula, quantifiers anywhere)

```
qelim_qf: LEMMA FORALL phi: FORALL m: qf?(qelim(phi, m))
qelim_ok: THEOREM FORALL phi: FORALL m, pt: length(pt) = m
  IMPLIES (fsem(qelim(phi, m))(pt) IFF fsem(phi)(pt))
```

Here `phi` is any first-order formula over the reals (`rcf_fol`'s `Fm`: `TRUE`, sign conditions on polynomials with rational coefficients, `NOT`, `AND` and `EXISTS`, from which the other connectives and `FORALL` are built), with its quantifiers anywhere, and m is any natural number, 0 included. `qelim` removes the quantifiers from the inside out: under each `EXISTS` the body is made quantifier-free first, and then that one quantifier is eliminated by `qe8`, with the variables in scope as free variables, or decided by `decide8` when there are none. No prenex form is needed. `gform_ok`'s `qe_g_ok` states the same for formulas written with PVS's own comparisons and connectives

($qe_g(g, m) = qelim(g2f(g), m)$), and `fod_qfd` (every first-order definable set is quantifier-free definable) follows from `qelim_ok`.

Theorem	What it says
<code>qe8_correct</code>	for every $m \geq 1$ and at least one quantifier: <code>qe8</code> 's answer holds at a point of $\mathbb{R}^m$ iff the quantified formula does; (<code>cad-qe</code>) instantiates it for prenex input under <code>IMPORTING pvs_cad</code>
<code>qe8_isqf</code>	the answer is always quantifier-free (the flag is always set)
<code>qe1cc_bf</code>	one free variable: <code>qe1cc</code> , over one Collins run, always answers, quantifier-free and equivalent (each sector of the free line described exactly, <code>epc_ok</code>)
<code>qe2cd_bf</code>	two or more free variables: over the Collins tower with derivative-closed free levels (<code>ctwd</code>), <code>qe2cd</code> always answers, quantifier-free and equivalent
<code>ents_sep</code>	Thom's lemma at work: when the free families are closed under the derivative, different cells always have different sign patterns
<code>qelim_ok</code>	every first-order formula, quantifiers anywhere, any $m \geq 0$: <code>qelim</code> 's answer, quantifier-free by <code>qelim_qf</code> , holds exactly where the formula does
<code>qe_g_ok</code>	the same for formulas written with PVS's comparisons and connectives (<code>gform_def</code>); (<code>cad-qe</code>) instantiates it for every formula its prenex route does not take (quantifiers inside connectives, binders over subtypes of the reals)
<code>qe_correct</code> , <code>qe_complete</code>	the statements of <code>qe8_correct</code> and <code>qe8_complete</code> for <code>qe</code> (<code>qe_all</code>), which runs the read-closure engine; (<code>cad-qe</code>) uses <code>qe</code> when the theory imports <code>qe_all</code> and not <code>qe8</code>
<code>qe2c_ev</code>	<code>qe</code> 's derivative-closed run succeeds for every large enough fuel parameter (the closure stops in a finite universe)

What you trust. The PVS kernel, which checks every step; PVS's ground evaluator, which runs `qe8`, `qe_g` or `qe` on the concrete input inside the proof (proof by reflection); NASALib's saved proofs of the theorems the library imports (from `reals`, `Sturm`, `Tarski`, `structures`, `analysis`, `complex`, `matrices`, `mult_poly` and `interval_arith`, and through them from other NASALib libraries such as `ints`, `power` and `trig`), which were not replayed here; and the reading of the statements: that `fsem` and `gsem` (the meaning of the quantified formula) and `qfsem` (the meaning of the answer) say what is intended. The procedures and the strategy code are not trusted; the procedures' correctness is the theorems.

Size. The quantifier elimination described here is 45 theories of the library (`cad/top.pvs`, 2 October 2026) with 550 proved formulas: the first procedure `qe` (26 theories, with `qe_all` and its examples `qe_cad_ex`), merged answers (`qe_mrg_def`, `qe_mrg`), `qe8` over the Collins run (13 theories, from `qe1c_def` to `qe8_ex`), `qelim` (`qelim_def`, `qelim_ok`) and formulas as PVS writes them (`gform_def`, `gform_ok`). It is built on the verified CAD library (cylindrical algebraic decomposition, sound and complete decision).

Using it in a proof: (`cad-qe`)

(`cad-qe FNUM`) takes a hypothesis or goal with at least one free real term, evaluates the elimination once inside PVS, prints the answer (`cad-qe: ...`), and replaces the formula by the answer. In a theory that imports `pvs_cad`, a prenex formula (one real per quantifier; grouped binders go through the nested form) is eliminated by `qe8` and the replacement is justified by `qe8_correct`; a formula of any other shape is eliminated by `qe_g`, justified by `qe_g_ok`. In a theory that imports `qe_all` and not `qe8` (as `qe_cad_ex` does), it evaluates `qe`, justified by `qe_correct`. Nothing is assumed. A real session (1 October 2026, `IMPORTING pvs_cad`; the printed `cad-qe`: line is one long line, wrapped here), proving that $t^2 + at + b = 0$ has a real root exactly when $a^2 \geq 4b$ (the direction " $\Rightarrow$ "):

```

w_quad.1 :
{-1}   EXISTS (t: real): t ^ 2 + a * t + b = 0
|-----
{1}   a ^ 2 >= 4 * b

Rule? (cad-qe -1)
cad-qe: ((b < 0) OR ((b = 0) OR ((b > 0) AND ((4*b + (-1)*a^2 = 0) OR
(4*b + (-1)*a^2 < 0)))))
length_null rewrites
length[list[mpoly]]((: :))
to 0
Eliminating the quantifiers of -1,
this simplifies to:
w_quad.1 :
{-1}   ((b < 0) OR
        ((b = 0) OR
         ((b > 0) AND
          ((4 * b + (-1) * a ^ 2 = 0) OR (4 * b + (-1) * a ^ 2 < 0)))))
|-----
{1}   a ^ 2 >= 4 * b

```

The quantifier is gone; what is left is plain arithmetic, finished with $a^2 \geq 0$ (3 s for the elimination). The other direction is the same command on the goal. The whole lemma is proved in the library: q8_quad in qe8_ex (through qe8) and qe_quad in qe_cad_ex (through qe).

Examples

Captured on 1 October 2026 in a theory that imports pvs_cad; every input here is prenex, so (cad-qe) eliminates by qe8. Answers are as (cad-qe) prints them, except that parentheses that do not change the reading (around atoms and around nested disjunctions) are dropped (the transcripts show them). “Reads as” is a hand simplification for the reader; the proved statement is about the printed answer. Times are wall-clock for the elimination step on an M-series Mac. **proved:** the IFF lemma is proved in the library, through qe8 in qe8_ex (q8_sqrt, q8_disc, q8_disc2, q8_eq2, q8_quad) and through qe in qe_cad_ex (qe_sqrt, qe_disc, qe_disc2, qe_eq2, qe_quad); the first row’s lemma, qe_win, only through qe. **one step:** the goal is proved by (cad-qe) alone; **step:** the elimination step is shown (it is a proved step; closing the arithmetic afterwards was not attempted).

Input	Answer, as printed	Reads as	Time
One free variable			
$\exists t \in [0, 10]. (5 - t)^2 + (t/2)^2 < D^2$ a moving point comes within D	$125/4 + (-25/4)*D^2 < 0$	$D^2 > 5$	3 s proved
$\exists t. t^2 = D$	$(-4)*D = 0 \text{ OR } (-4)*D < 0$	$D \geq 0$	2 s proved
$\forall x. x^2 + bx + 1 > 0$	$4 + (-1)*b^2 > 0$	$b^2 < 4$	2 s proved
$\exists x \exists y. x^2 + y^2 < D$ two bound variables	$(-256)*D < 0$	$D > 0$	2 s proved
$\exists x. x > 0 \wedge x^2 < c$	$(-1)*c < 0$	$c > 0$	2 s step
$\forall x \exists y. y^2 = x^2 + c$ alternating quantifiers	$(-256)*c = 0 \text{ OR } (-256)*c < 0$	$c \geq 0$	2 s step

Input	Answer, as printed	Reads as	Time
$\exists x. x^4 - 2x^2 = c$ degree 4	$(-64) + (-64)*c = 0$ OR $(-64) + (-64)*c < 0$	$c \geq -1$	2 s step
$\exists x. x^3 - 3x = c$	TRUE	always	2 s one step
Two free variables			
$\exists t. t = a \wedge t = b$	TRUE AND $(-1)*b + a = 0$	$a = b$	3 s proved
$\exists t. t^2 + at + b = 0$	$(b < 0)$ OR $(b = 0)$ OR $(b > 0$ AND $(4*b + (-1)*a^2 = 0$ OR $4*b + (-1)*a^2 < 0))$	$a^2 \geq 4b$	3 s proved
$\exists t. t^2 + at + b < 0$	$(b < 0)$ OR $(b = 0$ AND $4*b + (-1)*a^2 < 0)$ OR $(b > 0$ AND $4*b + (-1)*a^2 < 0)$	$a^2 > 4b$	3 s step
$\forall t. t^2 + at + b \geq 0$	$(b = 0$ AND $4*b + (-1)*a^2 = 0)$ OR $(b > 0$ AND $(4*b + (-1)*a^2 > 0$ OR $4*b + (-1)*a^2 = 0))$	$a^2 \leq 4b$	3 s step
$\exists t. t > 0 \wedge t^2 + at + b = 0$ a positive root	$(b < 0)$ OR $(b = 0$ AND $(-2)*a > 0)$ OR $(b > 0$ AND $((4*b + (-1)*a^2 < 0$ AND $(-2)*a > 0)$ OR $(4*b + (-1)*a^2 = 0$ AND $(-2)*a > 0)))$	$b < 0$, or $a < 0$ and $a^2 \geq 4b$	4 s step
Three free variables			
$\exists x. ax^2 + bx + c = 0$ the general quadratic	1,187 characters, below	solvable	8 s step

A positive root, in the prover

The same kind of session (the cad-qe: line wrapped):

```
s_pos.1 :
{-1} EXISTS (t: real): t > 0 AND t ^ 2 + a * t + b = 0
|-----
{1}   b < 0
{2}   (a < 0 AND a ^ 2 >= 4 * b)

Rule? (cad-qe -1)
cad-qe: (((b < 0) OR (((b = 0) AND ((-2)*a > 0)) OR ((b > 0) AND
      (((4*b + (-1)*a^2 < 0) AND ((-2)*a > 0)) OR ((4*b + (-1)*a^2 = 0) AND
      ((-2)*a > 0))))))
length_singleton rewrites
length[list[mpoly]]
  ((: (: mpol((: mpol((: mconst(0), mconst(1) :)) :)),
      mpol((: mconst(0), mconst(1) :)), mconst(1) :)) :))
to 1
Eliminating the quantifiers of -1,
this simplifies to:
s_pos.1 :
{-1}   ((b < 0) OR
      (((b = 0) AND ((-2) * a > 0)) OR
      ((b > 0) AND
      (((4 * b + (-1) * a ^ 2 < 0) AND ((-2) * a > 0)) OR
      ((4 * b + (-1) * a ^ 2 = 0) AND ((-2) * a > 0))))))
```

```

|-----
{1}   b < 0
{2}   (a < 0 AND a ^ 2 >= 4 * b)

```

Three free variables: the general quadratic

$\exists x. ax^2 + bx + c = 0$, eliminated in 8 s. The answer is correct but long: a disjunction of 19 cases over the cells of the decomposition (the sectors of c , then of b , then of a), each described by sign conditions on a , b , c and one more polynomial; as a whole it is equivalent to $(a \neq 0 \wedge b^2 \geq 4ac) \vee (a = 0 \wedge (b \neq 0 \vee c = 0))$. In the answer below (1,187 characters, captured with the table) that polynomial is printed $((-1)*b^2)*a + (4*c)*a^2$, that is, $4ca^2 - ab^2 = -a(b^2 - 4ac)$: minus a times the discriminant $b^2 - 4ac$.

```

((((c < 0) AND (((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b < 0) AND (a < 0))) OR ((((-1)*b^2)*a + (4*c)*a^2 > 0) AND ((b <
→ 0) AND (a < 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b < 0) AND (a = 0))) OR (((((-1)*b^2)*a + (4*c)*a^2
→ < 0) AND ((b < 0) AND (a > 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 < 0) AND ((b = 0) AND (a > 0))) OR
→ (((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b > 0) AND (a < 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 > 0) AND ((b > 0)
→ AND (a < 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b > 0) AND (a = 0))) OR ((((-1)*b^2)*a + (4*c)*a^2 <
→ 0) AND ((b > 0) AND (a > 0))))))))) OR (((c = 0) OR ((c > 0) AND (((((-1)*b^2)*a + (4*c)*a^2 > 0) AND ((b <
→ 0) AND (a < 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b < 0) AND (a = 0))) OR (((((-1)*b^2)*a + (4*c)*a^2
→ < 0) AND ((b < 0) AND (a > 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b < 0) AND (a > 0))) OR
→ (((((-1)*b^2)*a + (4*c)*a^2 > 0) AND ((b = 0) AND (a < 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 > 0) AND ((b > 0)
→ AND (a < 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b > 0) AND (a = 0))) OR (((((-1)*b^2)*a + (4*c)*a^2 <
→ 0) AND ((b > 0) AND (a > 0))) OR ((((-1)*b^2)*a + (4*c)*a^2 = 0) AND ((b > 0) AND (a > 0)))))))))

```

How it works

1. **One Collins run** on all the variables, free ones outermost (qe8). Its cells form a cylindrical algebraic decomposition with no certificate (col_cad_run); the walk over it, whose certificate is checked at run time and holds once its effort parameter is large enough (it is raised until then), gives the truth values at exact sample points. (qe runs the read-closure engine instead.)
2. **The outermost free variable** is cut into sectors (points and open intervals). Each sector gets an exact quantifier-free description: a rational endpoint is written directly, an irrational algebraic one by a polynomial that has it as its only root in a small interval (qe_ep; proved always possible, epc_ok).
3. **Over each sector**, the cells in the remaining free variables are given the truth value of the quantified part and the signs of the polynomials there. When equal signs mean equal truth values, the answer for the sector is the disjunction of the true sign patterns, over a subset of the polynomials that still tells the cells apart (chosen greedily). With one free variable the sectors themselves are these cells; when their signs do not tell them apart, the answer is the disjunction of the true sectors' exact descriptions. With one free variable, either way, the answer is replaced by its merged form, in which each run of consecutive true sectors is one interval, when that has fewer sign conditions (qe_mrg); with two or more free variables answers are not merged.
4. **When they do not** (two or more free variables), qe8 runs the Collins tower whose free families are closed under the formal derivative (ctwd). By Thom's lemma, different cells then always have different sign patterns (ents_sep), so the answer is always quantifier-free. (qe instead adds derivatives to the free families of its run until nothing new appears; that this stops is proved: every polynomial it can produce lies in one finite set fixed by the input.)
5. **Formulas of any shape** (qe_lim, qe_g): the quantifiers are removed from the inside out. Under each EXISTS

the body is made quantifier-free first; its sign atoms become the family of a prenex formula with one quantifier, which `qe8` eliminates (`decide8` decides it when no variable is free). `FORALL` is `NOT EXISTS NOT`.

Limits

- **Input shape.** First-order formulas over sign conditions on polynomials with rational coefficients. `qe` and `qe8` take prenex formulas with at least one quantifier and one or more free variables (`qe_complete`, `qe8_complete`). `qelim` (`qelim_ok`) takes every such formula, quantifiers anywhere, with any number of free variables, none included; it eliminates from the inside out, so no prenex conversion is needed. (`cad-qe`) needs at least one free real term. In a theory that imports `pvs_cad` (or `gform_ok`) it takes a formula of any shape through `qe_g` (`qe_g_ok`): quantifiers inside `NOT`, `AND`, `OR`, `IMPLIES` and `IFF` (grouped binders included there), and binders over `real`, `posreal`, `nnreal`, `negreal`, `npreal` and `nzreal`; a binder over any other type (`nat`, `int`, `rat`, `{x: real | ...}`) is refused. A grouped prenex formula goes through its nested form (`qe8`, or `qe` when only `qe_all` is imported), with either import; with only `qe_all` or `qe8` imported, (`cad-qe`) takes prenex formulas over `real`.
- **Answers are not minimal.** They are disjunctions over cells and sectors; the *a, b, c* example shows how long they get. Shortening them further is output polish, not correctness.
- **Cost** is the cost of the CAD run: 2 to 8 s for the examples here; runs with several bound and free variables together can take minutes (measured with `qe: QE_PLAN.md`, section 12).
- **Where it is.** In `cad/`; in the imports of `cad/top.pvs`, which describes every theory: `qe_ldd` to `qe_cad_ex` (`qe`, with `qe_mrg_def` and `qe_mrg`), `qe1c_def` to `qe8_ex` (`qe8`), `qelim_def`, `qelim_ok`, `gform_def`, `gform_ok`.